Calidad de datos del data lake

Validación, reconciliación entre capas y distribución de métricas

Javier Herrero Pérez
Whizdom.ai

15 de octubre de 2026

Versión 0.4

Resumen

Comprobé la calidad de los datos del lake de S3 en tres fases, todas sobre los datos completos (sin muestreo) y con informes que solo contienen agregados: (1) validar cada tabla contra un contrato con Great Expectations (GE), (2) reconciliar capas y tablas para encontrar registros duplicados o perdidos en los flujos de Kafka, y (3) estudiar la distribución de las 527 métricas de gold y sus bins. Los hallazgos principales (detalle en la tabla 6):

1. **Gold cuenta dos veces algunas ventanas de unos 10 minutos**, en todas las marcas a la vez: hasta un 1.6 % más de apuestas en un día (sección 4).
2. **El conector de Kafka a S3 escribió mensajes dos veces** (16 de julio y 4 de agosto, unos 1.1 millones de transacciones) **y perdió otros** (3 y 11 de septiembre, unas 140.000 transacciones).
3. **`gld_player_payments_daily` tiene un 0.25 % más de depósitos que la fuente**; la tabla de detalle de pagos sí cuadra.
4. **`gld_player_signals_daily` tiene 1 millón de filas de más** que las otras tablas diarias de jugador, el 81 % en junio y julio de 2026.
5. **Lo apostado en gold incluye las apuestas con importe positivo** (devoluciones o ajustes): si deben restar, el *turnover* está inflado un 0.5 %.
6. **Valores imposibles**: hasta 305 días de juego en un día, 57 millones en una escala de 0 a 100, y `brand_id` vacío en tablas enteras.
7. **El PSI del modelo de churn pierde bins en el 76 % de las métricas**, por la cantidad de ceros.
8. **Silver no existe en S3**, bronze se para el 2026-05-31 y del 1 al 17 de junio no hay transacciones en bruto en ninguna capa.

Índice

1. Alcance y punto de partida 2
2. Pipeline general 2
3. Fase 1: validación con Great Expectations 3
4. Fase 2: reconciliación 4
5. Fase 3: distribución de las métricas y bins 5

6. Hallazgos y preguntas abiertas 6

7. Estado y siguientes pasos 6

1. Alcance y punto de partida

Reglas de trabajo. Acceso de solo lectura a S3 con sesión SSO; cada número sale de todas las filas (SQL en DuckDB o un día completo en memoria); los informes versionados solo tienen conteos, porcentajes y estadísticos, nunca filas de jugadores; las columnas con datos personales no se cargan.

Qué hay en el lake. El inventario (sección 2) leyó solo listados y cabeceras de Parquet (tabla 1). Cada `tenant_id` de gold es una marca y cada marca es de un solo operador, así que landing se compara con gold por `brand_id`.

Tabla 1: Capas del lake (inventario del 2026-10-15).

Capa	Contenido
10-landing	Datos en bruto de Kafka, en vivo desde junio de 2026: 5 temas (transacciones, pagos, bonos, jugadores, planes de bono) para cada uno de los dos operadores, `primus` (13 marcas) y `secundus` (7 marcas). Unos 21 millones de transacciones al día.
20-bronze	Solo `transactions`: 8.330 millones de filas del 2025-04-01 al 2026-05-31; después se para. Tiene columnas de Kafka (partición y offset).
30-silver	Vacía.
40-gold	29 tablas de métricas y señales, una carpeta por día. Días sin fichero: `gld_player_bonuses` (262), `gld_player_payments` y `gld_player_payments_daily` (40).

2. Pipeline general

La figura 1 muestra el trabajo completo y la tabla 2 el comando de cada paso. El centro son los **contratos**: un YAML por tabla que dice cómo debe ser (columnas, tipos, grano, nulos, rangos, unidad, si es sumable, datos personales). Las tres fases leen los contratos y `configs/quality.yaml`. Lo que encuentra cada fase vuelve a `quality.yaml` como ajuste (un rango, un grano, un límite físico) y los contratos se regeneran.

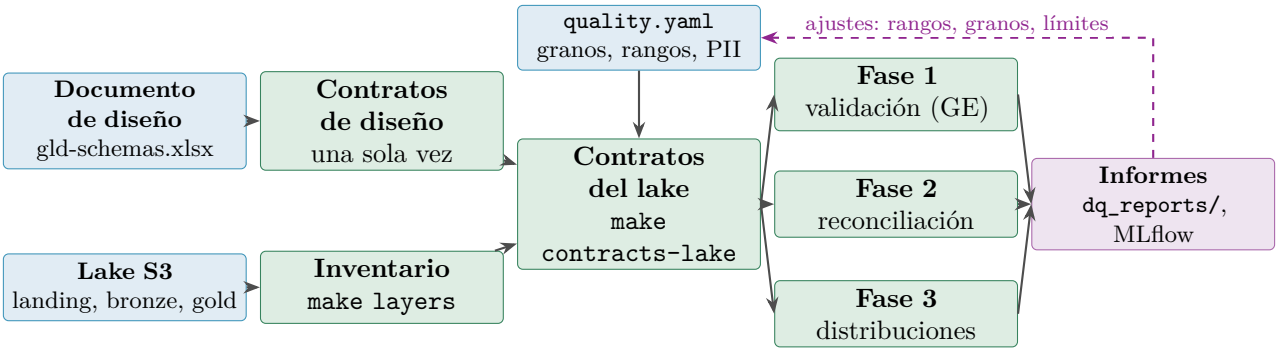


Figura 1: Pipeline general. Verde: pasos hechos. Azul: datos y configuración. Morado: salidas y ciclo de ajuste. Las tres fases leen los datos completos de S3 y solo escriben agregados.

Contratos. El documento de diseño describe el gold *futuro* y se va a borrar: lo convertí una vez a YAML y el script queda para justificar el origen de cada campo. Los contratos de las tablas reales (29 de gold, `transactions` de bronze y las 10 de landing) salen del inventario y toman del diseño, por nombre de columna, la unidad, el rango y si la medida es sumable. Granos, rangos corregidos y columnas personales se fijan en `quality.yaml`. Los 30 granos de bronze y gold están comprobados con datos reales; los de landing, todavía no.

Tabla 2: Pasos, comando y salida. Los pasos que leen S3 los ejecuto con mi sesión SSO.

Paso	Comando	Salida
Contratos de diseño	`src/quality/excel_to_contracts.py` (ejecutado una vez)	`configs/contracts/design/` (21)
Inventario	`make layers ROWS=1 [LAYERS=org/10-landing]`	`layer_inventory*.md` y `.json`
Contratos del lake	`make contracts-lake`	`configs/contracts/lake/` (40)
Fase 1	`make gx-suites`, `make validate-gx TABLES=... START=... END=...`	`gx_summary.md`, Data Docs (local)
Fase 2	`make reconcile START=... END=... [CHECKS=...]`	`reconciliation_<inicio>_<fin>.md`
Fase 3	`make distributions START=... END=...`	`distributions_<inicio>_<fin>.md`

3. Fase 1: validación con Great Expectations

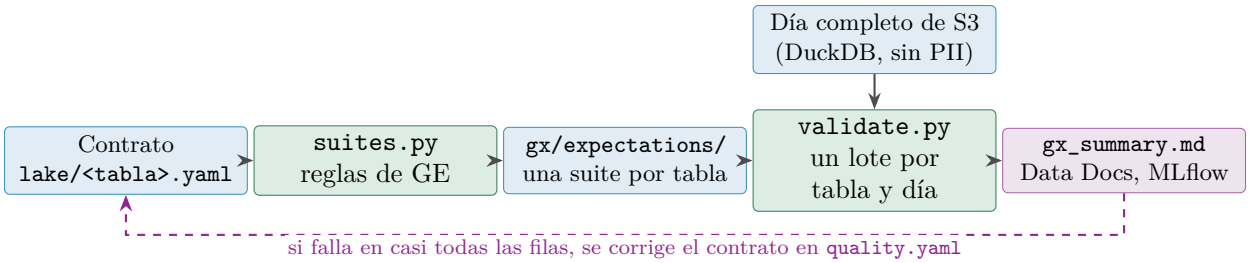


Figura 2: Pipeline de la fase 1. La caché por tabla y día hace que al repetir solo se validen días nuevos.

Reglas. Grano único; sin nulos en el grano; rango del contrato; importes y conteos de gold no negativos; la fecha de cada fila es la del fichero; columnas y tipos iguales al contrato (comparados con el footer, sin leer datos).

Ajustes al contrato. La primera validación separa los fallos del contrato (fallan casi todas las filas) de los fallos de datos (fallan pocas). Para decidir, el resumen da el mínimo y el máximo observados (tabla 3).

Tabla 3: Ajustes al contrato tras validar, y motivo.

Ajuste	Motivo
Puntuaciones de 6 tablas: rango 0 a 100	El diseño dice 0 a 1; el máximo observado es 100.
Grano de `gld_player_signals_daily` sin `brand_id`	`brand_id` está vacío en todas las filas.
Grano de `gld_player_gaming_daily` con `provider_code`	Sin él, 1.518 filas seguían repetidas con la última `_version`.
Entropías e índices de `gld_player_time_of_play`	Límites matemáticos: $\log_2 24$, $\log_2 7$, 4 (noche) y 3.5 (fin de semana).
Límites físicos (fase 3)	`play_secs_*` hasta los segundos del periodo, `value_decline` de 0 a 100, `wager_requirement` hasta 10^9.

Resultado. 3 días de mayo (3 tablas, 65 millones de filas) y 3 de octubre (29 tablas de gold, 53 millones de filas), cada ejecución en torno a un minuto: grano único en las 30 tablas, fechas y esquemas correctos, y 25 de 29 tablas de gold sin ningún fallo. Quedan tres problemas de datos: `brand_id` vacío

(100 % de `gld_player_signals_daily`, 31 % de `gld_player_lifetime`, 16 % de `gld_players`); ganancias negativas en pocas filas (hasta −176 EUR); y valores extremos (sección 5).

4. Fase 2: reconciliación

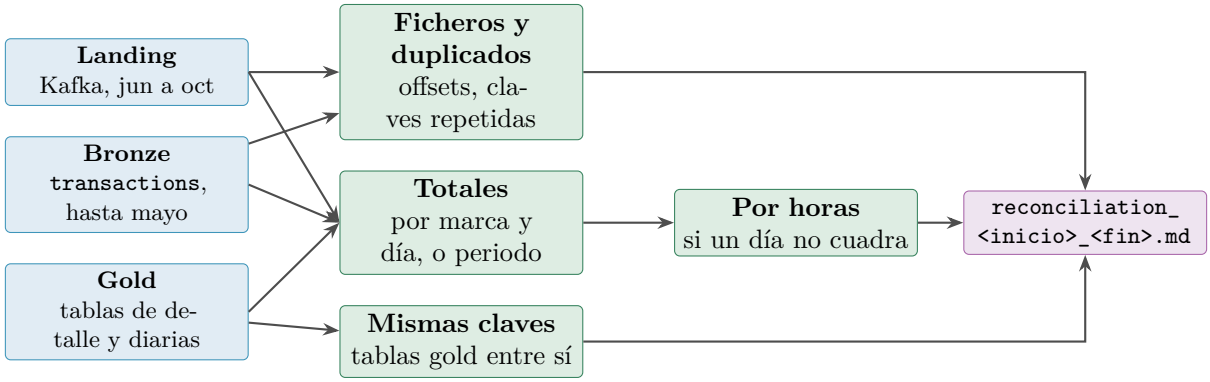


Figura 3: Pipeline de la fase 2. Todo es SQL en DuckDB que devuelve solo conteos. Cuando un total no cuadra, se baja de periodo a día y de día a hora.

Método. `reconcile.py` tiene cinco tipos de comprobación configurables en `quality.yaml`: offsets y claves repetidas de Kafka (`kafka`, `duplicates`), tablas que deben tener las mismas claves (`same_keys`), una tabla sumada frente a otra día a día (`totals`) y sobre todo el periodo (`period_totals`). Las claves que solo están en un lado se buscan también un día antes y después, para separar un desfase de fecha de una pérdida. Un importe cuenta como distinto si difiere más de un 0.1 %. El inventario comprueba además la continuidad de los ficheros de landing: cada nombre lleva la partición y el primer offset, así que cada fichero debe empezar donde acaba el anterior.

Tabla 4: Resultados de la reconciliación.

Comparación	Resultado
Bronze, Kafka (29 a 31 de mayo, 60 millones de mensajes)	Sin duplicados. Como mucho 2 mensajes perdidos. Bronze se cargó de golpe 22 días después del evento.
Bronze frente a gold (29 a 31 de mayo)	Todo llega: 17 apuestas de diferencia en 48.5 millones. Lo apostado cuadra (0.001 %) solo en valor absoluto: gold suma como apostado los `GAME_BET` con importe positivo. Lo ganado difiere un 0.13 %.
Ficheros de landing (junio a octubre)	16 de julio y 4 de agosto: el conector escribió dos veces unos 1.1 millones de transacciones y unos 68.000 pagos y bonos. 3 y 11 de septiembre: perdió unas 140.000 transacciones de `secundus`. Octubre, continuo. Los huecos pequeños de `players` encajan con un tema compactado. Sin duplicados por `uuid` (10 a 12 de octubre).
Landing frente a gold, pagos (1 a 12 de octubre)	Landing y `gld_player_payments` cuadran (502.792 frente a 502.793 depósitos). `gld_player_payments_daily` tiene un 0.25 % más de depósitos. Las retiradas pierden un 0.5 % del importe en gold.
Landing frente a gold, transacciones (1 a 12 de octubre)	7 días cuadran (0.01 % o menos); 5 días gold tiene un 0.55 % a 1.64 % más, en todas las marcas. No son eventos tardíos. Por horas, gold suma un 17 % más en dos horas de cada día analizado y en todas las marcas a la vez: unos 10 minutos contados dos veces, la firma de una reentrega de mensajes al proceso que carga gold.
Tablas diarias de jugador (2025-04 a 2026-10)	`signals_daily` tiene 1 millón de filas más que `risk`, `engagement` y `financial`, que cuadran entre sí (figura 4).
`gld_players` frente a `gld_player_lifetime`	El 47 % de diferencia son registrados sin actividad (no es error). Quedan unos 8.500 jugadores activos al día fuera de `lifetime` y un 0.03 % a 0.28 % con valores distintos.

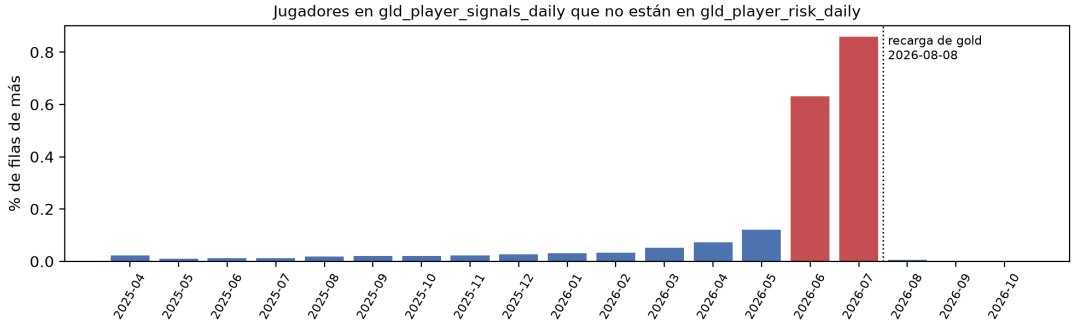


Figura 4: Filas de `gld_player_signals_daily` sin pareja en `gld_player_risk_daily`, por mes. La diferencia crece poco a poco hasta mayo de 2026, se dispara en junio y julio (81 % del total) y casi desaparece tras la recarga de gold del 2026-08-08.

5. Fase 3: distribución de las métricas y bins

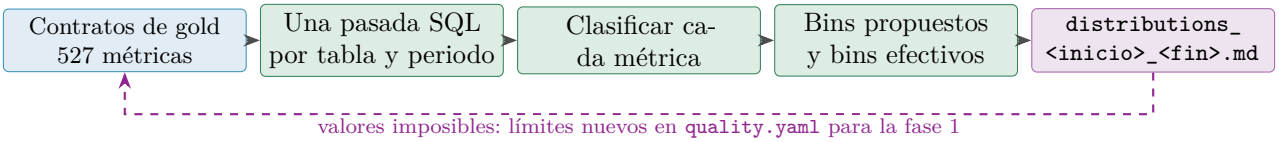


Figura 5: Pipeline de la fase 3: nulos, ceros, negativos, percentiles, asimetría, curtosis y valores distintos de cada métrica, sobre todas las filas del periodo.

Resultado (septiembre de 2026). Las métricas de gold están lejos de una distribución normal (tabla 5). Con deciles normales, como hoy en el PSI de las alertas del modelo de churn, **401 de 527 métricas (76 %) se quedan con menos de 10 bins**, porque los ceros repetidos se comen los intervalos; la figura 6 lo muestra con datos simulados. La propuesta para esas métricas es un bin propio para el 0 y cuantiles para el resto.

Tabla 5: Tipo de métrica y bins propuestos (umbrales provisionales en `quality.yaml`).

Tipo	Métricas	Bins propuestos
Más del 50 % de ceros	267	Un bin para el 0 y 9 cuantiles para el resto
Cola pesada (curtosis > 10)	95	Escala $\log(1 + x)$
Con signo (más del 1 % negativos)	63	Alrededor de 0, log con signo
Sesgada (asimetría > 1)	52	Deciles
Simétrica	27	Igual anchura (Freedman y Diaconis), como mucho 50
Diez valores o menos	23	Uno por valor

Valores imposibles. `play_secs_today` hasta 26.4 millones de segundos (305 días en un día); `value_decline` hasta 57 millones en una escala de 0 a 100; `wager_requirement` de 3×10^{18} (valor de relleno); `rtp_actual` hasta 491.000 y `pct_change` hasta 3.2 millones % (divisiones entre casi 0). Los tres primeros ya tienen límite en el contrato.

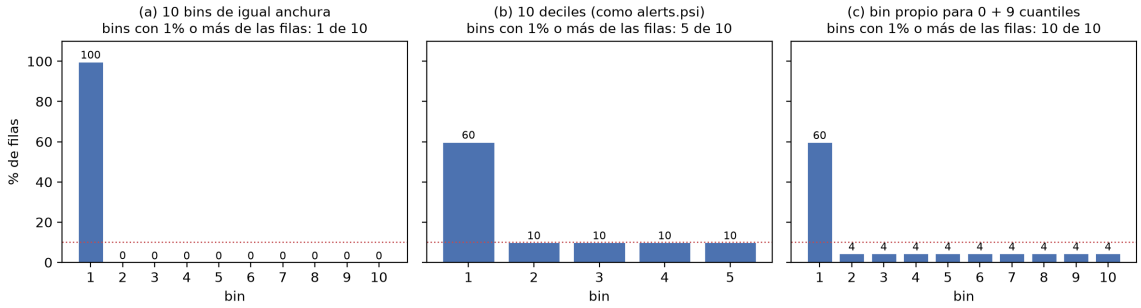


Figura 6: Tres formas de agrupar una métrica con un 60 % de ceros (datos simulados). Con deciles, como el PSI actual, solo quedan 5 bins útiles; con un bin para el 0 y cuantiles en el resto, los 10 tienen datos.

6. Hallazgos y preguntas abiertas

Tabla 6: Hallazgos, impacto y pregunta para el equipo de datos.

Hallazgo	**Impacto**	**Pregunta o acción**
Gold cuenta dos veces ventanas de unos 10 minutos	Hasta un 1.6 % más de apuestas en un día; afecta a apuestas, ganancias y tablas horarias	¿Hay reentregas al consumidor de Kafka que carga gold? ¿Es idempotente la carga?
Conector de Kafka a S3: duplicados (16 jul, 4 ago) y pérdidas (3 y 11 sep)	Landing no es una copia exacta de Kafka esos días	¿Qué pasó esos días? ¿Se puede recargar?
`gld_player_payments_daily` con un 0.25 % más de depósitos que la fuente	Depósitos diarios sobrestimados	¿De dónde salen? ¿Cómo se definen los intentos de pago?
`gld_player_signals_daily` con 1 millón de filas de más	Señales de jugadores que otras tablas no tienen	¿Qué jugadores incluye y por qué?
`GAME_BET` con importe positivo sumados como apostado	*Turnover* un 0.5 % más alto si son devoluciones	¿Deben sumar o restar?
`brand_id` vacío; valores imposibles	No se puede filtrar por marca; métricas con valores sin sentido	Corregir en el origen; ya vigilados por la fase 1
Silver vacía; bronze parado; hueco del 1 al 17 de junio	Sin datos en bruto para esos días	¿Dónde está silver? ¿Por qué se para bronze?
PSI con pocos bins	Alertas de deriva poco sensibles en el 76 % de las métricas	Cambiar los bins del PSI (acción mía)

7. Estado y siguientes pasos

Tabla 7: Estado del trabajo.

Parte	**Estado**	**Siguiente paso**
Contratos e inventario	hecho	Comprobar los granos de landing con datos.
Fase 1, validación	hecha	Validar más días con los límites nuevos.
Fase 2, reconciliación	casi hecha	Recorrer todas las horas de octubre y listar las ventanas contadas dos veces.
Fase 3, distribuciones	casi hecha	Repetir con la marca de valores extremos corregida; revisar el PSI del modelo de churn con su dataset de entrenamiento.
Documentación	hecha	Tutorial de GE en `GUIA_GREAT_EXPECTATIONS.md`.

Código. `src/quality/`: `layers.py`, `excel_to_contracts.py`, `inventory_to_contracts.py`, `contracts.py`, `suites.py`, `validate.py`, `reconcile.py` y `distributions.py`, con 16 tests que siembran errores en un lake falso y comprueban que se encuentran.

Correcciones del método. Durante el trabajo corregí cinco errores de mi código, todos cubiertos ahora por tests: la columna de detalle de la reconciliación mostraba solo el último día (las cifras de este documento ya son de todos los días); en las tablas horarias la hora se convertía en día antes de comprobar el grano; una resta de enteros sin signo desbordaba; la primera métrica de offsets medía el rango entero (un falso 97.9 % perdido); y la suma de pagos con clave corta multiplicaba filas.

Referencias

Apache Software Foundation (s.f.) *File format*, documentación de Apache Parquet. Disponible en: https://parquet.apache.org/docs/file-format/ (Consultado: 15 de octubre de 2026).

Great Expectations (s.f.) *GX Core documentation*. Disponible en: https://docs.greatexpectations.io/docs/core/introduction/ (Consultado: 15 de octubre de 2026).